CHAPTER 1

DESIGN A DEPLOYMENT SYSTEM

1 Introduction to Code Deployment System 2
2 Design of a Code Deployment System 8

Explore the essentials of automating deployment and understand the strategies to design a code deployment system.

1 Introduction to Code Deployment System

1.1 What is a code deployment system?

A code deployment system is a set of processes, tools, and techniques, often referred to as a **deployment pipeline** or **continuous deployment system**. It automates the distribution and deployment of software code across various environments, including testing, staging(A staging environment is a controlled environment that closely mimics the production environment. It is used to test and validate code changes, new features, updates, and configurations before they are deployed to the production environment where end-users will access them.), and production environments. Its main objective is to ensure that new code (or code changes) are smoothly and efficiently released from the development environment to the production environment while minimizing disruption and risk.
This lesson introduces the fundamentals of code deployment. We 'll learn key steps in the deployment process, explore common deployment strategies, and understand how to estimate resources and requirements when designing a deployment system.

Steps involved in the code deployment system

A typical deployment system follows these key stages to move code from development to production:

1. **Version control:** Code deployment process begins with version control, where developers utilize a version control system (such as Git) to manage and track changes made to the codebase.

2. **Continuous integration (CI):** In a continuous integration phase, code changes from different developers are frequently integrated into a shared repository. Preliminary automated tests(These tests are designed to quickly identify issues in code changes, ensuring that new code additions or modifications do not introduce errors or defects. Preliminary automated tests help catch problems early in the development process, making it easier and less costly to fix them.) are executed to ensure that the codebase remains stable and functional.

3. **Build phase:** After successful integration, a build process is triggered. During this phase, the system compiles the code, resolves dependencies, and generates deployable artifacts or executable files (binaries).

4. **Automated testing:** The deployment system runs extensive automated tests to check for any regressions or defects that the changes may inadvertently introduce to existing features. Different automated testsAutomatedTests are performed in this phase depending on the organization's needs and the nature of the software.

Quiz

Question 1: Question: How do automated tests in the deployment phase differ from those in CI?

- Deployment tests run only on developer machines.
- CI tests are optional, but deployment tests are mandatory.
- **[CORRECT]** CI tests validate code changes early, while deployment tests validate the full system before release.
- CI tests focus on performance and load testing.

5. **Staging environment:** The code changes are deployed to a staging environment that closely mimics the production environment. This enables additional testing and validation before the software is released to production.

6. **Deployment phase:** The code deployment system automatically deploys the tested and approved code changes to the production environment. This includes downloading code artifacts from the storage to each machine in the selected environment. This is often done gradually or using feature flags(Feature flags control whether or not an application's features or functionality are enabled or disabled. When we copy artifacts to our development environment, we can use feature flags to manage and control what features are turned on or off for testing and development.) to minimize the risk of potential issues.

7. **Monitoring and rollback:** After deployment, the system continuously monitors the application's performance and user feedback. If any issues are detected, a rollback plan is in place to revert to the previous stable version.

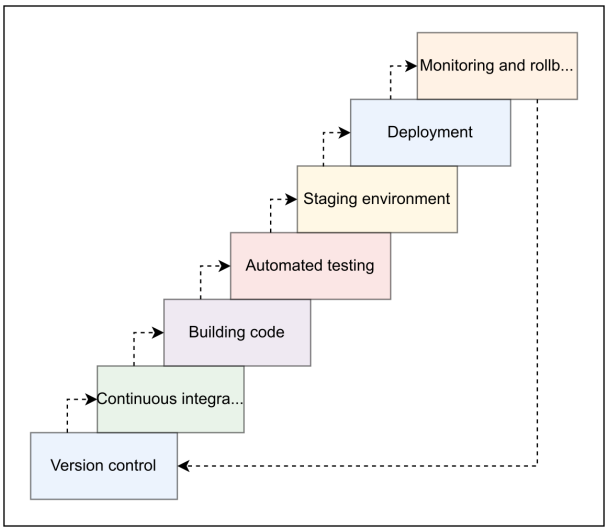


Figure 1.1: Steps involved in the code deployment system

Before designing the code deployment system, let's examine the various deployment strategies that influence the design.

Code deployment strategies

Code deployment strategies are techniques used to release new code versions into production while minimizing downtime, risk, and user disruption. They define the approach to rolling out updates to users rather than the operational steps of building and deploying code. Below are several widely used strategies:

- **Basic deployment:** Basic deployment involves directly releasing new code changes to the production environment. While simple, this approach can be risky, as it provides no safety net or rollback mechanism in case issues arise. It's generally unsuitable for large-scale or critical systems where reliability is essential.

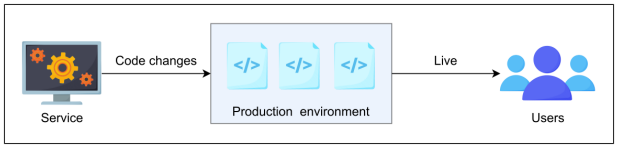


Figure 1.2: Releasing code directly to production

- **Multi-service deployment:** In multi-service deployment, multiple interconnected services are updated simultaneously as part of a coordinated release. This strategy ensures consistency among related services and prevents compatibility issues arising from mismatched versions. While it supports cross-service(Cross-service changes in deployment refer to modifications or updates that affect multiple interconnected services within a software application.) changes, coordinating the release of multiple services can be complex.

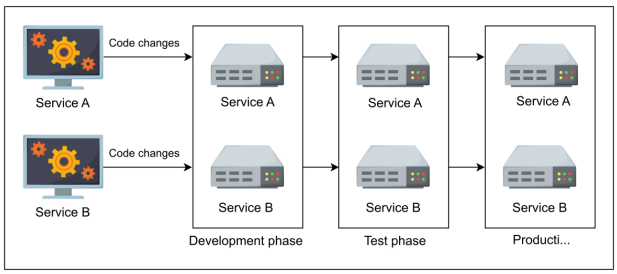


Figure 1.3: Releasing multiple dependent services together or in coordination to maintain system compatibility and stability

- **Rolling deployment:** Rolling deployment entails gradually deploying new code changes to a subset of servers while maintaining the current version on others. This minimizes downtime, ensures a gradual and controlled rollout, and distributes the load evenly across all services. However, slower than other strategies, they are straightforward to automate, which reduces the risk of widespread issues.

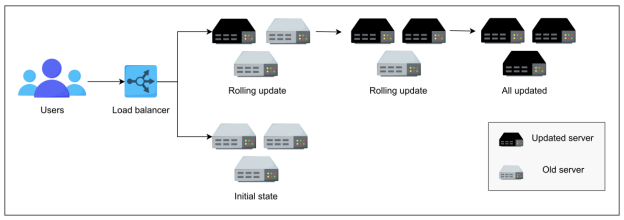


Figure 1.4: Updating servers in batches to avoid downtime

- **Blue-green deployment:** Blue-green deployment maintains two identical environments, one serving as the active production environment and the other for deploying new changes. This strategy offers minimal downtime, easy rollback, and a reduced risk of production issues. However, it requires duplicate infrastructure and introduces complexity in managing two separate environments.

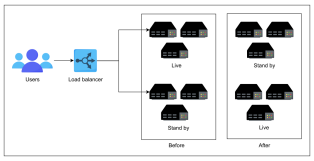


Figure 1.5: Switching traffic between two identical environments

- **Canary deployment:** This approach releases new code changes to a small subset of servers and opens them to some users (canaries) before deploying them to the entire fleet. This approach enables early bug detection, gradual rollout, and limited issue impact. It requires monitoring to ensure that canary users have a positive experience, and it can be complex to manage with different user segments.

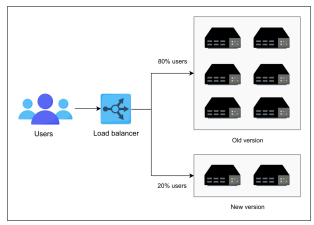


Figure 1.6: Gradually rolling out changes to a small group before full release

How would you decide which deployment strategy to use for a system that cannot afford any downtime? Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

Deployment strategies

Reference Answer:

Strategies like blue-green or canary deployment are preferable because they allow updates without interrupting the live system and provide quick rollback options.

We have explored different code deployment strategies. Now , let's shift focus to designing a system that can support these strategies and execute the deployment sequence effectively.

1.2 Requirements

Below are the functional and non-functional requirements of a code deployment system.

Functional requirements

Here are some essential functional requirements for a code deployment system:

- **Building code:** The system should be able to build the code within a designated timeframe, for example, 20 minutes. The compilation process might generate a binary file of several gigabytes.
- **Deploying code:** The system should be able to deploy code to many machines in several distinct worldwide regions.
- **Version control integration:** The system should be able to integrate with version control systems to fetch the latest code from repositories.
- **Environment configuration:** It should allow users to define and manage environment-specific configurations, such as database connections, API endpoints, and other settings.
- **Rollback:** The system should facilitate the ability to roll back to previous code versions in case of issues.
- **Deployment monitoring:** It should provide monitoring and reporting features to track the progress and success of deployments.

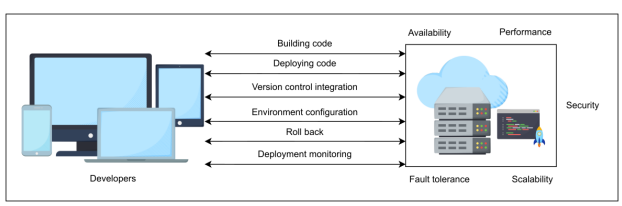


Figure 1.7: Functional and non-functional requirements of the code deployment system

Non-functional requirements

The following are some non-functional requirements for a code deployment system:

- **Availability:** The code deployment system should strive for high availability to minimize downtime, incorporating redundancy, failover mechanisms, and zero-downtime deployment strategies to ensure maximum reliability.
- **Fault tolerance:** Failures should be handled gracefully, with every build reaching `success` or `failure`.
- **Performance:** Deployments should start and finish within acceptable limits to minimize delays while meeting the required deployment rate.
- **Scalability:** The system should be able to handle increased load by scaling resources vertically or horizontally as required.
- **Security:** Sensitive data such as credentials and configuration data should be encrypted during transmission and storage. Similarly, every deployment action should be logged to maintain accountability.

Why is managing environment-specific configurations critical for a deployment system?
Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

Requirements

Reference Answer:

Managing environment-specific configurations is essential because each environment–development, staging, and production–requires different settings for security, performance, and integrations; keeping them separate prevents failures, avoids accidental misuse of production resources, and ensures reliable, predictable deployments.

Now, we'll estimate the required resources for our system based on the requirements.

1.3 Resource estimation

Assume that engineering teams across the globe deploy hundreds of services thousands of times each day. Each code build can take up to 20 minutes, generating numerous code artifacts. Based on this assumption, let's estimate the required resources in the following section:

Note: The 20-minute timeframe is a general guideline and not a strict requirement. Actual build times may vary depending on factors such as the size and complexity of the codebase, the available build infrastructure (e.g., caching, distributed compilation, or build farms), and the nature of the application being built (e.g., embedded systems may require longer build times compared to web applications).

Storage estimation

Let's estimate the amount of storage required based on the following assumptions:

- Number of applications (code updates): 200
- Frequency of deployments: 3000 per day
- Storage required for a single build: 20 GB
- The total amount of storage required to store binaries per day for all applications (code updates):

$$\text{Total storage} = 200 \times 3000 \times 20 \text{ GB} = 12 \text{ PB}$$

Note: This is a simplistic storage estimation based on deployment frequency and build size. In reality, factors like versioning, redundancy, backups, compression, and metadata could increase the required storage.

The following calculator computes the total amount of storage required per day:

Bandwidth estimation

A code build takes 20 minutes, followed by a 5-minute transfer to the regional blob store, considering network latency. With 2,000 machines per region, this 5-minute window is used to distribute the binaries across all machines. We can now estimate the required bandwidth to copy 20 GB binaries to each machine in 5 minutes, as follows:

$$\text{Bandwidth} = \frac{20 \text{ GB}}{5 \text{ minutes}} \approx 533.3 \text{ Mbps}$$

The number of servers estimation

To estimate the number of servers required, we consider the total number of deployments (3,000) per day. On average, a single server can perform up to 50 deployments per day. So , according to these numbers, the total number of servers required per day would be:

$$\text{Number of servers} = \text{Total deployments (per day)}\frac{}{\text{Server's deployment capacity}}$$
$$= 3000\frac{}{50=60 \text{ servers}}$$

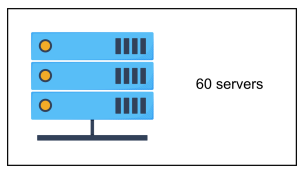


Figure 1.8: The number of servers required to perform 3000 deployments per day

Note: These time estimates reflect typical scenarios and can vary based on factors such as codebase complexity, infrastructure (e.g., build farms, caching, parallelism), deployment scale (e.g., number of services), and application type (e.g., microservices vs. monoliths). Environmental conditions, such as network speed and deployment tool performance, may also affect timing.

1.4 Building blocks we will use

The design of a code deployment system utilizes the following building blocks:

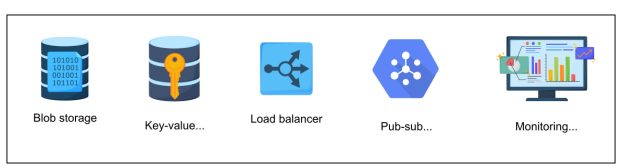


Figure 1.9: Building blocks required to design a code deployment system

- **Blob storage** is necessary for storing code binaries (artifacts) during the code build and deployment phases.
- **Key-value stores** are used to store the configuration information.
- **Load balancers** are used to distribute user requests among different servers and services.

- **A pub-sub system** is required to keep the code in the queue when the code build service is busy.
- **A monitoring service** checks the health of machines in the system.

In the next lesson, we'll focus on designing a code deployment system.

2 Design of a Code Deployment System

The previous lesson covered the fundamentals of the code deployment system: its requirements, resources, and key components. Now , let's examine the high-level design of the code deployment system.

2.1 High-level design

The high-level design of the code deployment system shows the various components we identified in the previous lesson. We have started developing a solution to meet the functional and non-functional requirements with the following design:

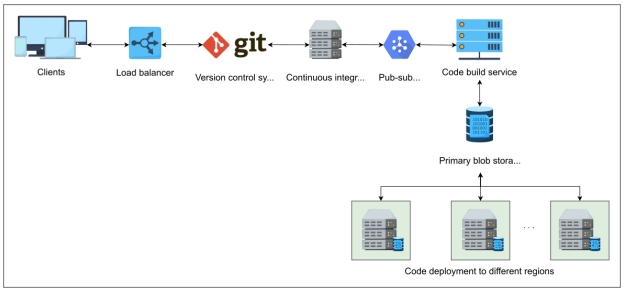


Figure 1.10: High-level design of the code deployment system

The workflow of the high-level design is shown below:

- Initially, users submit code to the version control system (VCS)(Tool for managing and tracking code changes across versions).
- The VCS triggers the Continuous Integration (CI(Platform that automatically builds and tests code changes after each commit)) service upon code changes. The CI system automates the process of integrating code changes from multiple developers into a shared repository.
- The code to build is placed in the pub-sub system. The code-build service takes code from the queue and converts it into a binary.
- The generated artifacts (binaries) are stored in the primary blob storage.
- Upon initiating deployment, the artifacts (binaries) from the blob storage are downloaded and installed on each machine in each region.

To better understand how the system operates at a functional level, let's explore the API design next.

2.2 API design

The following are essential APIs required for the code-building and deployment process:

Code-build APIs

Let's discuss some of the important APIs for the code-build process below:

- **Submit code:** This API is used to submit code to a specified repository in the VCS.

```
submitCode(code, repository, branch, message)
```

The `submitCode()` returns the commit hash uniquely identifying the new commit within the specified repository. This commit hash is used to reference and retrieve the commit later.
The following parameters are used for this API:

Parameter	Description
`code`	The code submitted to the VCS.
`repository`	This shows a path to a repository where the code is being submitted.
`branch`	Within a VCS, a branch represents an independent instance of the primary code or repository, typically identified by a unique name. In our case, it shows the branch name where the code is submitted or edited.
`message`	An optional message during a commit.

- **Trigger build:** This API triggers the process of building and preparing the codebase for deployment.

```
triggerBuild(project_id, branch, commit_hash, configuration)
```

The `triggerBuild()` API initiates the build process asynchronously. So , this API returns without waiting for the build process to complete. It returns a JSON object containing the URL to the queue where it is placed, and the build_id assigned to the build.

Parameter	Description
`project_id`	The ID of the project for which the build is being triggered.
`branch`	The branch name where the code is submitted.
`commit_hash`	A fixed-length hash value generated using SHA.
`configuration`	The specific settings, options, parameters, and environment variables that steer the course of the build process.

- **Access build artifacts:** This API is used to access the artifacts generated due to the build process.

```
accessBuildArtifacts(build_id)
```

Parameter	Description
`build_id`	A unique ID assigned to each build.

- **Debug build failures:** This API is used to troubleshoot and fix failures during the build process. When we call this API, it triggers a series of actions or processes within the deployment system. These actions may include retrieving detailed logs, error messages, and other diagnostic information related to the failed build. The API also provides diagnostic information that helps pinpoint the cause of the build failure. This information may include stack traces, error messages, code snippets, and relevant context.

```
debugBuildFailures(build_id)
```

Parameter	Description
`build_id`	A unique ID assigned to each build.

Code deployment APIs

Let's discuss some of the important APIs for the code deployment process below:

- **Select environment:** This API specifies the target environment (specific computing infrastructure) where the deployment is applied.

```
selectTargetEnvironment(environment)
```

Parameter	Description
`environment`	It is a string variable that shows the specific deployment environment. This variable can hold values such as `production`, `staging`, `development`, or any customer-specific environment corresponding to a particular configuration.

- **Initiate deployment:** This API initiates the deployment process to the specified environment.

```
initiateDeployment(build_id, build_version, strategy)
```

Parameter	Description
`build_id`	A unique ID assigned to each build.
`build_version`	A build version that is being deployed.
`strategy`	This object shows the deployment strategy and other configuration details to be applied during deployment.

- **Monitor deployment progress:** This API monitors the progress of the deployment process.

```
monitorDeploymentProgress(deployment_id)
```

- **View deployment logs:** This API is used to view the logs generated during deployment.

```
viewDeploymentLogs(deployment_id)
```

- **Manage rollback:** The API initiates rollback procedures in case of deployment issues.

```
manageRollback(deployment_id)
```

- **Validate and test:** This API triggers the validation and testing process on the deployed application, given the target environments.

```
validateAndTest(deployment_id, environment)
```

We've seen how services interact via APIs. Now, let's explore how data is structured in the storage schema.

Parameter	Description
`deployment_id`	A unique ID that is associated with each deployment.

2.3 Storage schema

Each of the above attributes in the API design requires support from the database—we'll need to store the details above in our storage schema to provide services to the API gateway. We also need to submit jobs(A job refers to a specific unit of work to be executed. In the context of a deployment system, compiling submitted code is treated as a job.) to the queue (pub-sub system). Our main concern is storing the job persistently in the queue to ensure the build process is reliable. Let 's look into the following essential storage schemas:

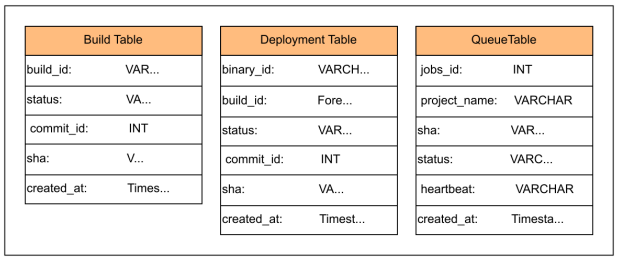


Figure 1.11: Storage schema for build, deployment, and the jobs in the queue

> **Quiz: Question 1:** How can we handle a scenario of the unexpected shutdown of a worker during the build process? **Answer:** In the 'QueueTable', we have an attribute 'Status', which shows various phases of the job as shown below: Queued: When a job is placed in the queue, its status is changed to queued. Running: This shows a job is in the running phase and has been assigned to the code build service for generating artifacts. Failed: This status shows the job failed due to some technical errors. Completed: The job is completed, and its artifacts are stored in the blob store.
>
> Canceled: The job is canceled. In addition to 'status', the table also includes a 'heartbeat' attribute. This attribute stores periodic heartbeat signals sent by the worker processing the job. The worker updates this attribute regularly to show it is active. If the heartbeat is not updated within the expected timeframe, the system assumes the worker has failed. The job status is then reset to 'Queued', making it eligible for reassignment to another worker.

After exploring the code deployment system's storage schema, let's focus on its detailed design, covering the internal components, their interactions, and the overall architecture.

2.4 Detailed design

The code deployment system comprises the following two phases:

- The code-building phase
- The code-deployment phase

The code-building phase

During the code-building phase, developers submit their code to a VCS such as GitHub. A CI system continuously monitors the repository for changes and is triggered whenever developers push updates or commit new code. Its primary purpose is to streamline the integration of changes from multiple contributors into a shared codebase. Beyond integration, the CI system also plays a crucial role in quality assurance by automatically running tests to catch potential issues early in the development cycle.

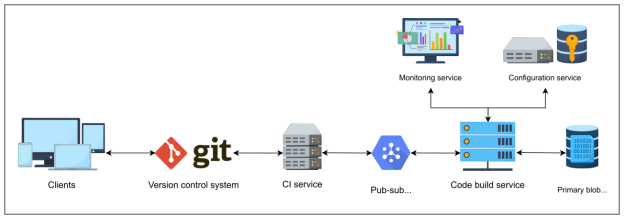
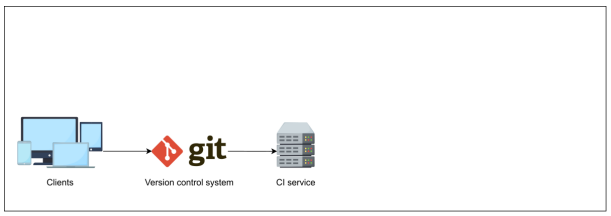


Figure 1.12: The code-building phase of the code deployment system

The following illustration presents the components involved in the code-build system:

The CI system is also responsible for triggering the build automation process in which the code is submitted to the pub-sub system. The available code-build server retrieves the code from the pub-sub system and begins building it. The code-build servers form a cluster for building artifacts. Here , the code is checked for the latest changes, dependency installation, and required libraries, particularly from project configuration files such as `package.json` or `requirements.txt`. It also compiles, transpiles(Transpiling, short for "source-to-source compilation," is the process of taking source code written in one programming language and converting it into equivalent code in another language.), or builds the code into executable formats (binaries), depending on the technology stack, and stores it in the primary blob storage.
Similarly, the build service may execute different tests, specifically unit and integration tests. The unit tests are executed to ensure that the code doesn't introduce regressions(Code regression refers to the unintended introduction of new bugs, errors, or issues into a software application that was previously functioning correctly.). On the other hand, integration tests verify that the newly built code works correctly when interacting with other components.
The configuration service manages the workers in the code-build service. It manages configuration information, synchronizes distributed processes, maintains distributed locks, and performs other related tasks. The primary focus of the configuration service is to provide reliability and availability while establishing coordination among the workers. Similarly , the monitoring service tracks the health, performance, and behavior of applications, systems, and networks. Monitoring services collect and analyze data from various sources, allowing administrators and teams to gain insights into the state of the code-build service.
The workflow of the code-building phase is illustrated below:

(a) Developers upload the code to a VCS such as GitHub



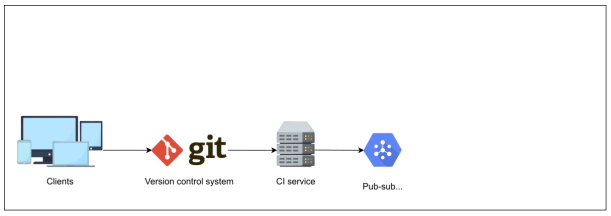
(b) The continuous integration system is activated whenever any changes are made to the VCS

We've walked through how the code build process operates. Now , let's move on to the next stage of the deployment system and see what happens after the build is complete.

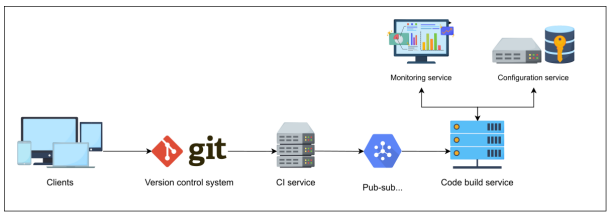
Code deployment phase

Deploying code involves several phases beyond code-building, each ensuring that new changes are successfully integrated into production systems while minimizing disruptions.
The following illustration shows the detailed design of the code deployment phase:

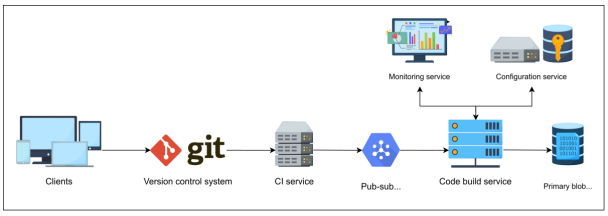
When the primary blob storage is updated with the binaries of the new code, the status in the queue is changed to *Completed*. The replication service updates the regional blob storage with the new binary build.



(a) The continuous integration system triggers the build process and the code is submitted to the pub-sub system



(b) The available workers take the code from the pub-sub system and start building it. The monitoring service keeps a check on the health of each worker, while the configuration service manages the workers in the build process



(a) When the code is built, it is stored in the primary blob storage for distribution to different regions when the deployment process is triggered

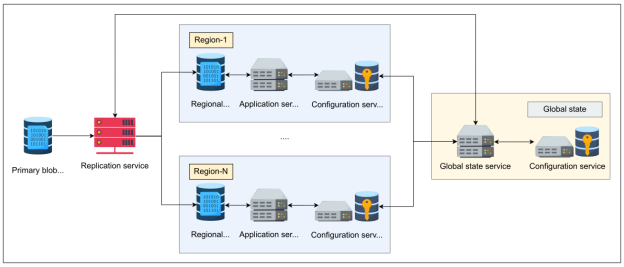


Figure 1.16: The deployment phase of the code deployment system

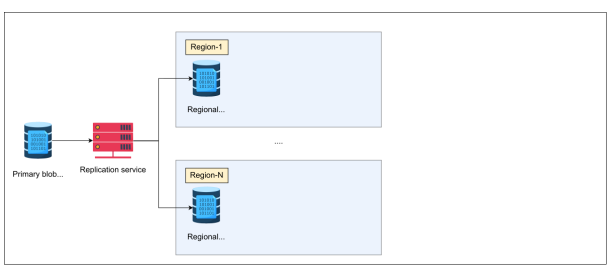
Earlier , we assumed that the minimum size of a single build is several gigabytes. Therefore, downloading the large file from the regional blob store to each machine over the network would take a considerable amount of time. We assume that the application servers in a region are connected via a peer-to-peer network. This will help decrease the binaries' downloading time to each machine.

> **Quiz: Question 1:** How does the performance differ when copying large binaries using the client-server paradigm as opposed to peer-to-peer (P2P)? **Answer:** The difference in performance between copying big binaries using client-server and P2P is significant and depends on the architecture and network properties. In client-server architectures, a central server distributes binary files to multiple clients. The server becomes a bottleneck, particularly under high load, affecting overall performance.
>
> In P2P architectures, the workload is distributed among multiple peers.
>
> This means that transfers can be made in parallel and more efficiently.
>
> P2P can take advantage of the upload or download capabilities of all the peers involved. This can lead to faster data transfer speeds, especially when the network has latency or bandwidth constraints.

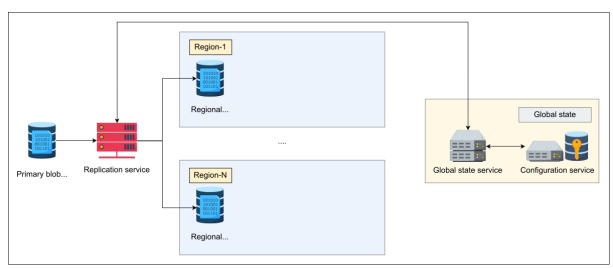
After the regional blob stores are populated with the latest binaries, the system updates a global state service that tracks deployment readiness across regions. This marks the binaries as ready for deployment throughout the system.
To coordinate deployments, a key-value store keeps the desired version information. When a deployment is triggered, the global configuration is updated with the target version (e.g., `build:1.1`), as shown below:

```
{ build_version: build:1.1
}
```

Regional configuration services continuously check the global state for updates. Once a new version is detected, each regional service notifies its local application servers to fetch the corresponding binaries from their regional blob store. The servers then download and install the new version, completing the deployment process.
The configuration service in each region also ensures that code changes are seamlessly integrated into the production environment.



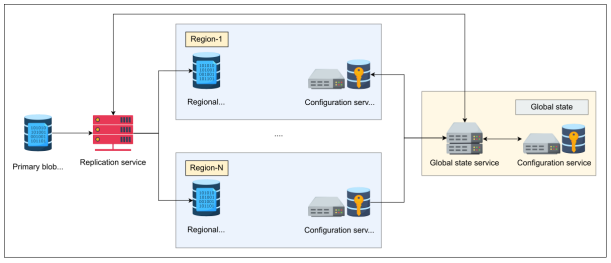
(a) When the code build status is changed to "Completed" the replication service updates the regional blob storage with the new build



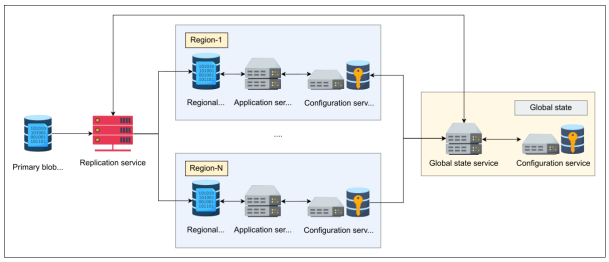
(b) The global state service is updated when the replication to each regional blob stores is completed. The binaries are now deployable

2.5 Putting everything together

Let's put the build and deployment process together in the following figure. Once the code-build phase is completed, the replication service updates the regional blob storage with the newest build. Similarly , the global state configuration service is updated with the newest build version during deployment. The regional configuration service updates the newest build version and instructs the application server to download binaries from the updated regional blob storage. Once the download is completed to the regional blob storage, it is installed on each application server, and the application status is set to "up". This completes the deployment cycle.



(a) Upon initiating the deploy command, the global state configuration service updates the regional configuration service



(b) The application servers continuously poll the regional configuration service for any changes. When any change is made to the configuration service, the application servers fetch the binaries from regional blob storage and install them

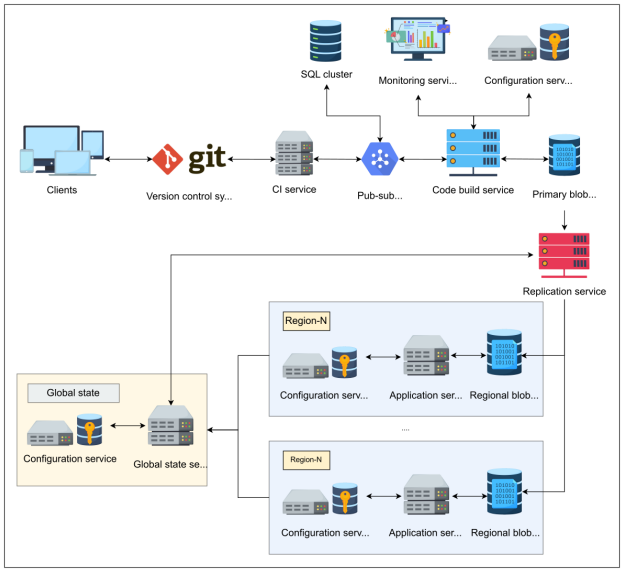


Figure 1.19: Detailed design of the code deployment system

2.6 Requirements compliance

We will see how the proposed design of the code deployment system satisfies both the functional and non-functional requirements essential for reliable, scalable, and observable software delivery across global infrastructure.

Achieving functional requirements

We will start by revisiting the functional requirements as mentioned in the previous lesson and examining how our code deployment systemaddresses them:

Functional Requirements	Techniques
Building code	• The CI system detects code changes and triggers the build process. • The code-build service, coordinated via the pub-sub system, compiles and packages the code into binary artifacts (several GBs). • The compiled output is stored in primary blob storage to provide a durable and centralized location accessible by all deployment components
Deploying code	• The replication service copies binaries from the primary to the regional blob storage. • Application servers in each region download binaries via peer-to-peer networks. • Regional configuration services coordinate the update to local application servers.
Version control integration	• The system integrates with Git-based version control to enable the CI system to pull the latest changes from repositories for build and deployment.
Environment configuration	• A central configuration service maintains global and regional configuration states. • Regional configuration services fetch and apply environment-specific settings to application servers.
Roll back	• The configuration service can update the deployment version (e.g., build:1.0) to a previous one tracked by the global state service. • Application servers can re-fetch and install older binaries from blob storage.
Deploying monitoring	• The monitoring service tracks the health, status, and performance of the build and deployment pipeline, offering visibility into failures or delays.

We have explored how the system design addresses functional requirements. Now let's see how the system meets non-functional requirements.

Fulfilling non-functional requirements

Beyond core functionality, a code deployment system must be fault-tolerant, scalable, and secure to support continuous delivery at a global scale. The table below summarizes how the design meets these critical non-functional requirements:

Non-functional Requirements	**Techniques**
Availability	• The configuration service enables automated failover by rerouting build requests to backups. • Redundancy is ensured with duplicated servers, databases, and load balancers.
Fault tolerance	• The monitoring system checks system health and notifies administrators. • Workers regularly send heartbeats through the pub-sub system, recorded in an SQL database; if one is missed, the task is reassigned to an available worker.
Performance	• Builds are executed asynchronously to avoid blocking. • Binaries are distributed across regions. • Peer-to-peer network within each region enables fast replication.
Scalability	• Queue service auto-scales with build request volume. • Configuration service deploys more servers as needed. • System resources dynamically adjust based on load.
Security	• Multi-factor authentication (MFA) and OAuth are enforced. • Data is encrypted at rest and in transit. • Secure coding practices, firewalls, and intrusion detection are used. • IAM service handles access control and credential management and enforces the least privilege.

We've explored the non-functional requirements and the techniques used to achieve them. Let's test our understanding with a quiz.

Quiz

Question 1: What mechanism is established to ensure system availability during the build phase?

- OAuth authentication
- **[CORRECT]** Redundancy through duplicated elements
  Explanation: Redundancy of resources enhances the system's availability. If one component fails, the other is on standby to provide the desired services.
- Elastic resources management
- Blue-green deployment strategy

Question 2: How does the queue service handle scalability during the build phase?

- Through OAuth authentication
- **[CORRECT]** Through elastic resources and auto-scaling
  Explanation: Elastic resource management adds or removes resources based on the current load, which enhances the scalability of the system.
- Through the blue-green deployment strategy
- Through intrusion detection and firewalls

Question 3: How does the code deployment system address the challenge of a failing worker during the build phase for fault tolerance?

- Multi-factor authentication (MFA)
- Auto-scaling the configuration service
- Regular vulnerability management
- **[CORRECT]** Heartbeat mechanism and job reassignment
  Explanation: The heartbeat mechanism ensures a worker is up and performing the build process. If a worker fails to send a heartbeat in the designated time, it is considered failed, and the build job is assigned to another worker.

We've seen the detailed design of the code deployment system. Now, let's explore how a deployment strategy impacts the System Design:

2.7 Impact of deployment strategies on the CDS

The deployment strategy adopted in a code deployment system (CDS) plays a significant role in shaping its architectural decisions. In our proposed design, we rely on a basic deployment strategy, which is flexible and straightforward to implement. While simple, this strategy requires careful handling of version consistency and rollback mechanisms to ensure reliability during the rollout process.
Different deployment strategies, such as blue-green or canary deployments, have distinct requirements that

must be accommodated within the System's Design. For instance, a blue-green deployment necessitates the creation of duplicate environments, while a canary deployment demands mechanisms to roll out changes to a subset of users gradually.
The choice of deployment strategy can significantly impact requirements such as redundancy, load balancing, version management, and monitoring mechanisms. Therefore, a code deployment system should be designed to be flexible and modular. It should support various strategies tailored to project needs, ensuring performance, reliability, and scalability.
What trade-offs exist between performance and reliability when implementing a blue-green deployment?
Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

Performance vs. reliability

Reference Answer:

A blue-green deployment improves reliability by allowing instant rollback and zero-downtime releases, but it introduces performance trade-offs because you must run two full production environments simultaneously, which doubles infrastructure usage and can strain resources. Traffic switching also risks cache cold-starts, uneven load distribution, or momentary latency spikes as the new environment warms up. In short, you gain safer, more reliable releases at the cost of higher resource consumption and potential short-term performance dips.

2.8 Conclusion

Designing a scalable and resilient code deployment system depends on various organizational and architectural decisions. This chapter presents an approach to building a flexible deployment system, focusing on the code-building process and a basic deployment strategy. Since deployment can follow multiple strategies, choosing a specific one can significantly influence the system's overall design. Supporting components such as VCS and CI are acknowledged but not explored in depth due to their inherent complexity. Finally , the chapter discusses various methods for meeting requirements and examines how different deployment strategies affect the overall design of the code deployment system.